

Caso Práctico Cloud

Prueba técnica de habilidades cloud

Definición

El candidato debe demostrar conocimientos sobre los servicios de AWS así como de herramientas para CI/CD como pipelines, github actions, terraform, cloudformation, etc.

Indicaciones

- Cada ejercicio debe contar con su documentación, explicando de que trata, estructura, recursos necesarios y modo de ejecución
- Se debe incluir todo material adicional que considere necesario para la correcta evaluación
- El entregable debe ser mediante un repositorio de git
- Se evaluará la capacidad de crear recursos de forma generica, escalable y mantenible
- Se considerará todos los aspectos explicados y aplicados para la optimización de costos

Caso 1:

Crear una forma de automatizar la creación de lambdas a modo template para reducir el tiempo de atención a otros miembros del equipo. Las características solicitadas para este ejemplo son:

- Tiempo de ejecución: Python 3.13
- Arquitectura: arm64
- Estructura del código:

```
1 /
2 |   {lambda_name}
3 |   |   lambda_function.py
```

- Código que debe contener:

```
1 import json
2
3 def lambda_handler(event, context):
4     # TODO implement
5     return {
6         'statusCode': 200,
7         'body': json.dumps('Hello from Lambda!')
8     }
```

Caso 2:

Habilitar pipelines con github action preferentemente para permitir subir cambios a lambdas, considerando 3 ambientes identificados por alias

- dev

- staging
- prod

es importante considerar que cada ambiente va a contar con variables de entorno diferentes por ejemplo

- redis_host
- db_host

es importante contar con mecanismos de rollback optimizados

Caso 3:

Dado un proyecto construido en php con laravel, con vue se requiere crear un mecanismo de despliegue continuo que incluya los pasos:

- ejecución de pruebas
- obtener dependencias composer
- obtener dependencias node
- compilacion
- empaquetado
- despliegue

Es importante indicar que servicios se recomiendan utilizar para este despliegue, justificando su uso, así como buscar optimizar cada paso, reduciendo los tiempos de deploy y rollback

Caso 4:

Dada la siguiente estructura de proyecto

```
1 cloud-lab
2 |
3 |--- app
4 |   |--- main.py
5 |   |--- requirements.txt
6 |   |--- test_main.py
7 |
8 |--- .github
9 |   |--- workflows
10 |       |--- deploy.yml
11 |
12 |--- README.md
```

con los siguientes fragmentos de código:

app/main.py

```
1 import json
2
3 def handler(event, context):
4     name = event.get("name", "world")
5
6     return {
7         "statusCode": 200,
8         "body": json.dumps({
```

```
9         "message": f"Hello {name}"
10     })
11 }
```

app/requirements.txt

```
1 boto3==1.42.62
```

app/test_main.py

```
1 from main import handler
2
3 def test_lambda():
4     response = handler({"name": "test"}, None)
5     assert response["statusCode"] == 200
```

.github/workflows/deploy.yml

```
1 name: Deploy Lambda
2
3 on:
4   push:
5     branches:
6       - main
7
8 jobs:
9
10  deploy:
11    runs-on: ubuntu-latest
12
13    steps:
14
15      - uses: actions/checkout@v3
16
17      - name: Install dependencies
18        run: pip install -r app/requirements.txt
19
20      - name: Run tests
21        run: pytest
22
23      - name: Package Lambda
24        run: zip function.zip app/*
25
26      - name: Deploy
27        run: aws lambda update-function-code \
28              --function-name interview-lambda \
29              --zip-file fileb://function.zip
```

Realiza las modificaciones necesarias para el correcto despliegue en AWS, incluye uso de terraform

Caso 5:

Indica procesos de revisión, herramientas a utilizar y procesos de propuesta para realizar la optimización de costos mediante el siguiente escenario.

Una empresa tiene:

- 10 EC2 t3.large
- RDS PostgreSQL db.m5.large
- S3 con 20 TB
- CloudFront

Costo mensual: **\$6,000 USD**

Caso 6:

Crear un template (Terraform o CloudFormation) que despliegue:

- 2 Lambda
- 1 API Gateway
- 1 DynamoDB
- 1 SQS
- permisos IAM necesarios

El objetivo de esta solicitud es crear la infraestructura necesaria para un webhook que necesita dar respuesta en menos de 2 segundos, pero el procesamiento de la información recibida se realice de forma asíncrona ya que para procesar los datos se requiere al menos 17 segundos, se debe tener histórico de ejecuciones y mostrar métodos de monitoreo para el correcto uso.